HTTPS

Candado del navegador

6 DE OCTUBRE DE 2026

YAZMIN RODRIGUEZ SALAZAR

1. Introducción

El protocolo HTTP transmite la información en texto plano, lo que permite que cualquier observador en la red intercepte peticiones, encabezados y datos sensibles. HTTPS soluciona este problema encapsulando HTTP dentro de un canal cifrado mediante TLS (Transport Layer Security), el cual garantiza tres propiedades fundamentales: confidencialidad (solo cliente y servidor leen los datos), integridad (los mensajes no se alteran en tránsito sin detectarse) y autenticación (verificación de la identidad del servidor mediante certificados digitales).

2. Ejercicio 1 y 2: Generación y lectura del certificado digital

Se generó una llave privada RSA de 2048 bits (llave.pem) y un certificado autofirmado válido por 365 días (certificado.pem) mediante OpenSSL. Al inspeccionar el certificado con el comando openssl x509 -in certificado.pem -text -noout, se obtuvieron los siguientes datos:

- **Emisor (Issuer):** CN = localhost
- **Sujeto (Subject):** CN = localhost
- **Vigencia (Validity):**
  - **Not Before:** Nov 6 2026 (Fecha de generación del certificado)
  - **Not After:** Nov 6 2027 (365 días de validez)
- **Nombre Alternativo del Sujeto (X509v3 Subject Alternative Name):** DNS:localhost, IP Address:127.0.0.1

3. Ejercicio 3 y 4: Servidor HTTPS y advertencia del navegador

Se levantó el servidor HTTPS en el puerto 8443 envolviendo el socket del servidor con ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER) y cargando la cadena de certificación (certificado.pem y llave.pem).

¿Aparece el candado? ¿Con qué matiz?

No aparece el candado cerrado tradicional. En su lugar, el navegador muestra un indicador de alerta ("No es seguro" o un ícono de advertencia/candado tachado) indicando que, aunque la conexión está cifrada mediante TLS, la autoridad que emitió el certificado no es válida ni reconocida.

4. Ejercicio 5: Reflexión sobre el certificado autofirmado

- **Si el tráfico ya va cifrado con el certificado autofirmado, ¿por qué el navegador insiste en la advertencia?**

Porque TLS no solo promete cifrado (confidencialidad e integridad), sino también **autenticación**. Un certificado autofirmado cifra los datos exactamente igual que uno comercial, pero como está firmado por nosotros mismos y no por una Autoridad de Certificación (CA) incluida en la lista de confianza del navegador (como Let's Encrypt o DigiCert), el navegador no tiene forma de comprobar que el servidor no sea un impostor realizando un ataque de intermediario.

- **¿Qué tendría que pasar para que no apareciera la advertencia?**

En un entorno en internet con un dominio público, el certificado tendría que ser emitido y firmado por una Autoridad de Certificación (CA) reconocida. En un entorno de laboratorio local (localhost), tendríamos que importar manualmente nuestro archivo certificado.pem dentro del almacén de "Autoridades de certificación raíz de confianza" del sistema operativo o del navegador.

5. Ejercicio 6 (Punto Extra): Bitácora CRUD sobre HTTPS

Se integró el módulo ssl al servidor CRUD de la Práctica 8 (servidor_bitacora.py) y se realizaron las pruebas con curl y con el cliente requests en Python.

Comportamiento de las tres pruebas con curl:

curl --cacert certificado.pem https://localhost:8443/incidencias: Funciona correctamente y de forma segura, ya que le proporcionamos explícitamente nuestro certificado para validar la identidad del servidor.

curl -k https://localhost:8443/incidencias: Funciona y devuelve los datos cifrados, pero desactiva la validación de autenticidad del certificado.

curl http://localhost:8443/incidencias: Falla porque intenta enviar una petición HTTP en texto plano hacia un puerto que espera iniciar una negociación criptográfica (TLS Handshake).

- **¿Qué diferencia hay entre -k y --cacert?**

El parámetro --cacert mantiene la seguridad completa de TLS al indicarle al cliente en qué certificado específico debe confiar para autenticar al servidor. Por el contrario, -k (equivalente a verify=False en requests) apaga por completo la verificación del certificado, aceptando cualquier certificado sin validar quién es el servidor y dejando la puerta abierta a ataques de suplantación de identidad.

- **¿Por qué el código del CRUD (los métodos, los contadores y el archivo incidencias.json) no necesitó ningún cambio?**

Porque TLS funciona en una capa inferior a la de aplicación (envolviendo el socket de red mediante wrap_socket). El cifrado y descifrado ocurre de manera transparente en el canal de comunicación antes de que los datos lleguen a la lógica de BaseHTTPRequestHandler, por lo que los métodos do_GET, do_POST, do_PATCH, do_PUT y do_DELETE reciben y procesan peticiones HTTP estándar exactamente igual que antes.

6. Cuestionario Final (Sección 6)

1. **Un sitio con certificado válido y candado, ¿garantiza que el sitio es confiable? ¿Qué garantiza exactamente?**

No garantiza que el sitio sea benigno, honesto o seguro en sus intenciones (un atacante puede crear un sitio falso de phishing con un dominio propio y obtener un certificado válido gratuito en minutos). El candado garantiza únicamente tres cosas técnicas: que los datos viajan cifrados entre el navegador y ese servidor (**confidencialidad**), que no fueron modificados en el trayecto (**integridad**) y que el servidor realmente controla el nombre de

dominio que aparece en la barra de direcciones según una CA reconocida (**autenticación**).

2. **¿Dónde han visto certificados rotos o expirados en la vida real? ¿Qué hicieron?**

Es común encontrarlos en portales internos de universidades o dependencias gubernamentales cuando vence la fecha de renovación de su certificado, en interfaces de administración web de routers domésticos o switches (que usan certificados autofirmados por IP privada), y en portales cautivos de redes Wi-Fi públicas. En sitios públicos la medida correcta es no continuar ni introducir credenciales; en equipos de red locales propios donde se conoce el dispositivo, se verifica la dirección IP antes de acceder.

3. **Su proyecto integrador: si su sistema manejará credenciales o datos sensibles, ¿qué implicaciones tiene servirlo por HTTP plano?**

Servir un sistema con credenciales mediante HTTP plano expone toda la plataforma a ataques de captura de tráfico (*sniffing*) con herramientas como Wireshark. Cualquier persona conectada al mismo segmento de red o Wi-Fi podría leer en texto claro los nombres de usuario, contraseñas, cookies o tokens de sesión, permitiendo el robo inmediato de cuentas, la violación de privacidad de los usuarios y la alteración maliciosa de las respuestas en tránsito.